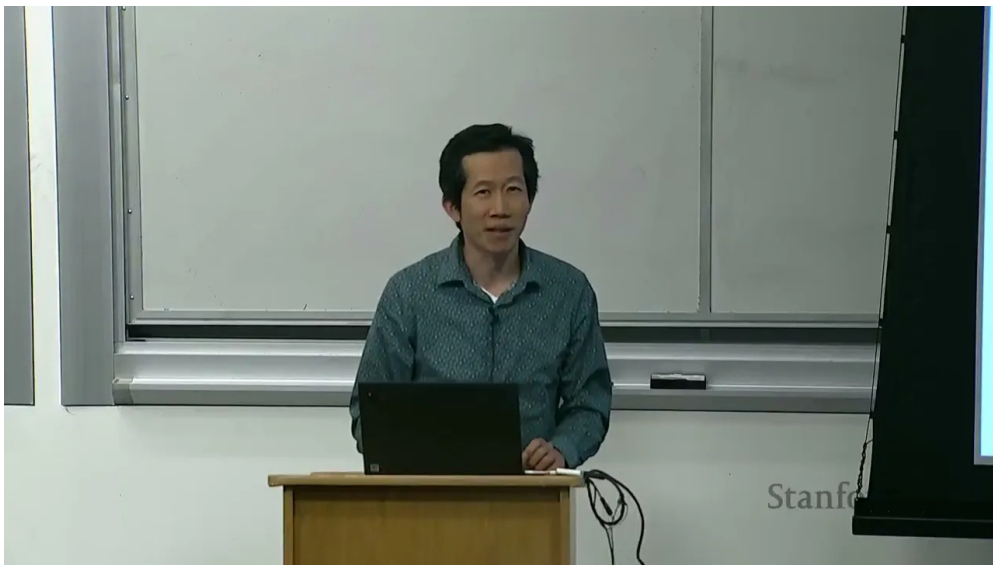


课程笔记

CS336 Lecture 13: Data

基于讲者授课内容整理

2025 年春季



视频作者/频道: Stanford CS336

发布日期: 2025-03

视频时长: -

视频链接: <https://stanford-cs336.github.io/spring2025/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：数据不是附属品	3
1.1 为什么公司对数据这么保密	3
1.2 数据也是长尾问题	4
2 训练阶段与数据对象	4
3 预训练语料：高质量小集合到大规模网页	5
3.1 BERT 的书籍和 Wikipedia	5
3.2 Wikipedia 很好，但并不完整	6
3.3 数据污染和 poisoning	6
3.4 GPT-2 的 WebText 和 OpenWebText	6
3.5 Common Crawl：更接近互联网，但远不等于互联网	7
3.5.1 Worked Example: Common Crawl 的规模感知	7
3.6 CCNet: Wikipedia 风格的质量代理	8
3.6.1 Worked Example: 基于 Perplexity 的质量过滤	9
3.7 C4: 规则化清洗的典型代表	9
3.7.1 Worked Example: 去重方法对比	10
3.8 GPT-3、The Pile、Gopher、LLaMA：数据配方开始变成公开工程	10
3.8.1 主要预训练数据集对比	11
3.8.2 数据过滤方法对比	12
3.9 The Pile 的社区意义	12
3.10 RedPajama、RefinedWeb、FineWeb：网页数据进入精加工时代	13
3.11 Dolma、DCLM、Nemotron-CC：从多样性到可比较过滤	13
3.12 为什么 open source 之后，配方会越来越像工程规范	14
3.13 代码数据的特殊地位	14
3.14 预训练数据本章小结	15
4 中训练、后训练与能力定制	15
4.1 长上下文不是顺带出现的	15
4.2 从任务到 prompt	16
4.3 Instruction tuning 和 chat data	17
4.3.1 后训练数据策略对比	18
4.4 为什么会越来越依赖合成数据	18
4.5 中训练与后训练本章小结	19
5 法律、许可与 fair use	19
5.1 知识产权法基础	19
5.2 为什么版权会卡住数据	20
5.3 许可、CC 协议和训练授权	20
5.4 fair use 的四个因素	20
5.5 TOS 为什么也重要	21
5.6 拓展阅读	22

5.7	把整堂课串起来：一个端到端的数据管线	22
5.8	一个更具体的例子：把网页做成训练集	23
6	总结与延伸	23
6.1	本讲的核心问题链	24
6.2	拓展阅读	24

1 导言：数据不是附属品

这一讲不是在补一个“数据集名单”，而是在回答一个更根本的问题：**语言模型到底靠什么长出来**。前几讲已经讲完了架构、优化器、tokenizer、并行和 scaling laws，默认前提都是“数据已给定”。这一讲把前提翻过来，讨论数据从哪里来、怎么过滤、怎么混合、怎么做成训练集，以及哪些法律和合规边界不能绕过。

这一讲的核心判断

1. 数据不是训练流水线里的一个附件，而是决定模型上限的主要变量。
2. 数据工作是长期、并行、可拆分的工程，不是一次性的抓取任务。
3. 好模型通常不是因为“多看了互联网”，而是因为“更会挑数据”。
4. 数据处理不仅是技术问题，也是版权、隐私、平台规则与供应链问题。

讲者对” data is the most important thing” 这个判断非常强。理由并不抽象：架构往往由少数人定义，数据工作却可以拆成大规模并行流水线，持续调优、持续清洗、持续重配比。模型越大，数据质量、覆盖面和过滤策略就越直接地决定算力是否用在刀刃上。

Data is the New Moat

在大模型竞争中，架构设计趋同（几乎所有主流模型都是 decoder-only Transformer），训练框架也日趋标准化。真正构成差异化壁垒的，是**数据**：你能拿到什么数据、怎么清洗、怎么配比、怎么持续迭代。这就是为什么 Llama 3 的技术报告详细描述了架构和训练流程，却对数据只给出模糊描述——数据配方本身就是最核心的竞争力。对创业公司而言，如果没有独特的数据获取和处理能力，仅靠公开权重或公开代码很难建立持久优势。

1.1 为什么公司对数据这么保密

公开模型论文通常会详细讲结构、训练流程和损失函数，但对数据往往只给粗粒度描述。原因有两个：

- **竞争原因**：数据配方本身就是产品竞争力。
- **法律风险**：版权、隐私和条款违约都可能引发争议。

这也解释了为什么”公开权重”并不等于”公开数据”。前者解决模型可用性，后者涉及数据供应链和合规问题。

公开程度	典型做法	背后的考量
公开权重	发布 checkpoint，任何人可下载推理	推广生态、吸引开发者
公开架构	论文详细描述层数、注意力头、激活函数	学术声誉、可复现性
数据常隐藏	只给粗粒度描述(如”网页 + 书籍”)	竞争壁垒 + 法律风险规避

表 1: 公开论文里最透明和最不透明的，往往不是同一件事

1.2 数据也是长尾问题

为什么数据是 long-tail problem

你可以雇很多人、开很多爬虫、写很多过滤规则，但世界上真正稀有、边缘、昂贵、合规复杂的数据，总是以长尾方式分布。架构是少数设计点，数据却是无数细粒度选择的叠加。

当你要做一个既懂代码、又懂数学、又会对话、还要能长上下文和安全拒答的模型时，没有哪一种单独数据源可以包打天下。数据质量与覆盖面之间存在天然张力：高质量来源（如 Wikipedia、教科书）通常覆盖面窄；广覆盖的来源（如随机网页）噪声极大。成熟的数据工程，本质上就是在这条光谱上持续做折中。

2 训练阶段与数据对象

数据工作通常不止一阶段。比较清晰的划分是：

阶段	目标	典型数据
Pre-training	学到语言分布与通用知识	大规模原始网页、书籍、百科、代码、论文
Mid-training	纠偏和补强某些能力	更高质量网页、数学、代码、长上下文、合成任务
Post-training	对齐成可对话、可指令跟随的模型	指令数据、聊天数据、偏好数据、RL 数据

表 2: 语言模型训练常见分阶段

Base model 与 Instruct model

Base model 通常指 pre-training 加上 mid-training 后的检查点。

Instruct/chat model 通常指 post-training 后的模型，也就是能更自然地跟人对话的版本。

讲者强调，阶段边界在现实里并不总是严格清晰。很多模型会插入更多子阶段，但方向都一样：先用大规模低质量数据打底，再用更小规模高质量数据修正和对齐。以 AI2 的 OLMo 为例，其三阶段分别使用了不同的数据集（预训练用 Dolma，mid-training 用 Dolmino Mix，后训练用 Tulu 数据），每阶段的数据规模和质量要求差异显著。

阶段	主要信号	常见风险	工程重点
Pre-training	语言续写、事实共现、风格吸收	噪声大、重复多、合规复杂	先把规模做起来，再谈更精细的能力
Mid-training	数学、代码、长文档、网页精选	过拟合少数任务、分布偏移	用更高质量数据把能力补齐
Post-training	指令响应、对话、偏好	幻觉、拒答不稳、风格失控	把模型变成可用的助手，而不是只会续写的文本引擎

表 3: 三阶段训练的信号、风险与目标并不相同

很多人会把“把数据喂得更干净”理解成同一件事，但实际上每个阶段的目标函数都不一样。

pre-training 更像让模型见识世界的分布，mid-training 更像补课和纠偏，post-training 则是在控制输出方式和交互方式。若把三者混成一个大混合桶，最后通常只会得到一个既不稳定、也不容易解释的模型。

从数据规模看，三个阶段的量级差异极大：

阶段	典型 token 量级	数据来源	质量要求
Pre-training	1T – 36T tokens	网页、书籍、代码、百科	覆盖面优先，噪声可接受
Mid-training	100B – 1T tokens	精选网页、数学、代码	质量明显高于预训练
Post-training	10K – 1M 样本	指令、对话、偏好标注	极高质量，逐条审核

表 4: 三阶段训练的数据规模差异可达 3-4 个数量级

3 预训练语料：高质量小集合到大规模网页

3.1 BERT 的书籍和 Wikipedia

一个早期但典型的例子是 BERT。它的训练数据由两部分构成：

- **BooksCorpus**：来自 Smashwords 的自出版电子书，规模约 7 千本、接近 10 亿词。
- **Wikipedia**：自由编辑的百科全书，提供相对高质量的通用文本。

这个组合说明了早期预训练数据的思路：先找高质量、结构化、可长期复用的文本，再把它们拼成足够大的训练集。

来源	优点	缺点	典型用途
BooksCorpus	连贯、长程依赖强、语体自然	授权风险高、来源不透明	早期语言建模、长句子结构学习
Wikipedia	结构化、引用明确、事实密度高	覆盖面窄、语体偏百科	知识锚点、质量代理、过滤参考
Web text	覆盖广、包含真实用户语言	噪声和重复极多	大规模预训练、长尾补充

表 5: 书籍、百科和网页在数据工程中的角色完全不同

从教学角度看，这一页的关键不是”哪种数据最好”，而是”哪种数据在什么阶段最合适”。书籍和百科适合建立稳定的语言骨架，网页适合补足分布覆盖，后训练数据则负责把骨架变成能对话、能遵循指令的交互系统。

BERT 的一个重要设计选择是用**文档级**而非句子级的输入。这和早期的 1 Billion Word Benchmark（句子打散）形成对比——连续文档能让模型学到跨句依赖和篇章结构。

BooksCorpus 的一个历史教训

看起来“能抓到”不等于“能自由使用”。BooksCorpus 后来因为违反 Smashwords 的服务条款而被下架。数据工程从一开始就不是纯技术问题。

3.2 Wikipedia 很好，但并不完整

Wikipedia 的价值在于结构化、带引用、编辑机制强，但它也有明显边界：不包含原创观点、宣传性内容、个人网页和大量具体经验知识。它非常适合做知识锚点，却不可能覆盖世界的全部表达。截至 2024 年，英文 Wikipedia 有超过 6200 万篇文章覆盖 329 个语言版本，但其收录标准（基于“关注度”原则：需要有可靠来源的显著报道）天然排除了大量长尾知识。

Wikipedia 的现实优势

- 结构稳定，质量较高。
- 有编辑和引用机制。
- 是高质量网页语料的重要“正样本”。
- 很多质量过滤器都把它当成目标分布或代理分布。

3.3 数据污染和 poisoning

讲者特别提醒：即使是 Wikipedia 这种高质量来源，也不能默认安全。数据污染研究已经展示，攻击者可以利用周期性 dump 的时间窗口插入恶意编辑，随后再回滚，从而让错误内容进入训练数据。具体攻击流程如下：

1. 攻击者在 Wikipedia dump 生成前短时间内插入恶意编辑（例如，将某品牌关联到负面情感）。
2. 周期性 dump（通常每隔数周）抓取了包含恶意编辑的快照。
3. 恶意编辑随后被 Wikipedia 管理员回滚，页面恢复正常。
4. 但训练集中已经保留了被污染的版本，模型可能学到错误关联。

数据安全的核心教训

训练数据的风险不只来自“脏网页”，也来自“看起来很干净、但可被操纵”的来源。数据源越权威，越不能掉以轻心。

3.4 GPT-2 的 WebText 和 OpenWebText

GPT-2 的 WebText 思路很代表性：从 Reddit 中高 karma 帖子的外链网页出发，用“人类实际愿意转发什么”作为粗糙质量代理。具体来说，WebText 收集了 Reddit 上获得 3 个以上 karma 的帖子中的外链网页，共 800 万页、约 40GB 文本。开源复现版本 OpenWebText 再通过 URL 抽取、英文过滤（使用 Facebook 的 fastText 分类器）和近重复删除来构建类似数据集。

WebText 的处理流水线可以总结为四步：

1. **社交过滤**：只保留 Reddit karma ≥ 3 的帖子中的外链 URL。
2. **URL 抽取**：从帖子元数据中提取所有外链网页地址。
3. **语言过滤**：用 fastText 分类器过滤掉非英文页面。
4. **近重复删除**：移除内容高度相似的页面，避免冗余。

WebText 的方法论意义

它不是在证明 Reddit 等于高质量，而是在证明一个更现实的点：**当全网太大、太乱时，人类行为信号可以作为粗糙的质量代理。**

3.5 Common Crawl: 更接近互联网，但远不等于互联网

Common Crawl 是整个网页语料生态的底层原料。它每月进行一次大规模爬取，产物通常经历 WARC 到 WET 再到可训练纯文本的多级转换。讲者特别强调，HTML 到文本的转换不是无关紧要的前处理，转换策略会显著影响下游效果。

步骤	在做什么	为什么重要
抓取 frontier	从种子站点扩展到新 URL	决定覆盖面和新鲜度
保存 WARC	保留原始 HTTP 响应	便于回溯和复现
抽取 WET	从 HTML 中提取可读文本	下游模型主要消费的是文本而不是 DOM
去重过滤	去掉重复页面和模板页	避免 token 被低价值重复内容吞掉
语言/质量打分	挑出更像自然语言的段落	降低噪声和垃圾页比例

表 6: Common Crawl 到训练文本之间的中间步骤不是“可有可无”的

WARC、WET 这些格式名字听起来只是文件容器，但它们背后代表的是数据责任边界：你保留多少原始信息、什么时候丢弃 HTML 结构、怎么记录可复现性，都会决定后续过滤器还能不能追查问题来源。对大规模训练来说，日志、版本和可回滚能力并不比模型超参次要。

3.5.1 Worked Example: Common Crawl 的规模感知

要理解 Common Crawl 对预训练的意义，需要先对其规模有直觉：

指标	数值
历史爬取次数 (2008–2025)	约 100 次
单次爬取时长	10–12 天，使用约 100 台机器 (2016 年数据)
种子 URL 数量	至少数亿个
单次快照原始大小	数十 TB (WARC 格式)
C4 的起点 (2019 年 4 月快照)	约 1.4T tokens (原始)
C4 过滤后	156B tokens (约 11% 保留率)
FineWeb (95 个快照合并)	15T tokens
DCLM-pool (多快照合并)	240T tokens

表 7: Common Crawl 的规模：从原始爬取到可训练文本，保留率通常在 5%–15%

从 1.4T tokens 到 156B tokens，C4 的过滤保留率约 11%。这意味着 **将近 90% 的原始网页内容被判定为不适合训练**。到了更激进的过滤方案（如 DCLM-baseline 或 FineWebEdu），保留率可以低至 1%–5%。这个数字本身就说明了数据清洗的核心地位。

网页语料的三个现实问题

1. 网页噪声非常大，绝大多数内容不是训练语言模型的好材料。
2. 相同内容会以很多近重复版本出现，必须做去重。
3. 语言识别和质量过滤通常依赖启发式或分类器，而不是“直接相信网页”。

为什么 Web 数据需要 URL 级别的去重

很多人以为去重只需要在文档内容层面做（比如比较文本哈希值），但实际上 **URL 级别的去重同样关键**。原因有三：

- **同一 URL 跨快照重复**：Common Crawl 每月爬取一次，同一个 URL 可能在不同快照中反复出现，且内容几乎不变（如企业主页、产品说明页）。如果不做 URL 去重就直接合并多个快照，这些页面会以倍数膨胀。
- **URL 规范化问题**：`http://` 和 `https://`、带 `www.` 和不带、URL 参数顺序不同——这些“不同 URL”实际上指向完全相同的内容。
- **镜像站和内容农场**：大量网站会原封不动复制其他站点的内容，URL 不同但内容完全一致。仅靠 URL 去重不够，还需要结合内容指纹（如 MinHash）进行模糊去重。

FineWeb 等现代数据集会先做 URL 过滤（去掉已知低质量域名、色情站点、spam 域），再在内容层面做 MinHash 去重，形成两级防线。

讲者还提到一个重要细节：HTML 到文本的转换工具选择会显著影响下游效果。DCLM 论文的实验表明，直接使用 Common Crawl 提供的 WET 文件（其转换质量较低）比使用 `trafilatura` 或 `resiliparse` 等专业工具从 WARC 重新提取文本，在下游任务上可以差好几个点。这意味着“前处理”远非无关紧要的琐事。

3.6 CCNet: Wikipedia 风格的质量代理

在 Common Crawl 之上，各种后续数据集做了不同程度的清洗。CCNet 的核心思路是：先去重，再做语言识别，最后用 Wikipedia 训练出的 n -gram 模型做质量过滤。CCNet 对低资源语言特别有价值——它的目标不仅是清洗英文网页，还包括为乌尔都语等低资源语言提供高质量预训练数据。

CCNet 的三步流水线：

1. **段落级去重**：对文本做轻量规范化后，移除重复段落。
2. **语言识别**：使用 `fastText` 分类器识别语言，只保留目标语言。
3. **质量过滤**：用 Wikipedia 训练的 KenLM 5-gram 模型打分，保留“看起来像 Wikipedia”的文档。

CCNet 体现了什么

它体现的是一种非常普遍的数据工程方法：你先定义一个“像好数据的样子”，再把海量候选文本往这个方向过滤。

3.6.1 Worked Example: 基于 Perplexity 的质量过滤

CCNet 使用的 Wikipedia perplexity 过滤是理解数据质量分类器的最好入口。其训练过程如下：

1. **正例**：取 Wikipedia 的全部文本作为”高质量”的代理分布。
2. **训练语言模型**：在 Wikipedia 文本上训练一个 KenLM 5-gram 语言模型。
3. **计算 perplexity**：对 Common Crawl 中的每个文档，用该语言模型计算 perplexity。
4. **过滤**：perplexity 越低，说明该文档越”像 Wikipedia”，越可能是高质量文本。按 perplexity 分桶，取最低的若干桶作为训练数据。

直觉上，一段写得通顺、信息密集、结构清晰的文本，在 Wikipedia 训练的语言模型下 perplexity 较低；而一段充满广告、重复短语和语法错误的网页，perplexity 则会很高。

质量过滤的偏见问题

以 Wikipedia 作为”高质量”代理分布，隐含着一个重要假设：Wikipedia 的写作风格代表了”好文本”。但这个假设并不总是成立：

- **方言与口语**：非标准英语（如 African American Vernacular English）在 Wikipedia 中几乎不出现，perplexity 过滤会系统性地排除这类文本。
- **特定社区**：Reddit 的技术讨论、StackOverflow 的问答、医学论坛的患者叙述——这些对训练有价值的文本，其风格与 Wikipedia 差异很大。
- **创意写作**：诗歌、小说片段、实验性写作的 perplexity 可能很高，但它们对语言模型的表达能力至关重要。

RefinedWeb 的作者正是出于这个原因，选择了基于规则的过滤而非基于分类器的过滤——他们认为 ML 过滤器会引入难以察觉的系统性偏见。

3.7 C4: 规则化清洗的典型代表

C4 是 T5 论文里最重要的数据贡献之一。它使用大量手工规则清洗 Common Crawl，比如只保留以标点结尾、且足够长的行，过滤少句页面、脏词、占位符和 boilerplate，并只保留英语。具体规则包括：

- 只保留以标点符号结尾且包含 ≥ 5 个单词的行。
- 删除少于 3 个句子的页面。
- 删除包含任何”脏词”的页面（基于公开的脏词列表）。
- 删除包含 { 的页面（排除代码）、包含 lorem ipsum 的页面、包含 terms of use 的页面等。
- 使用 langdetect 过滤，只保留英语概率 > 0.99 的页面。

起点是 2019 年 4 月的 Common Crawl 快照（约 1.4T tokens），经过上述规则过滤后得到 806GB 文本（约 156B tokens）。

规则清洗的代价

规则简单意味着透明，但也意味着误杀和漏网都很多。它能保留一些不太像 Wikipedia、但很有用的句子，也会放进一些语义上并不理想的内容。例如，”包含 { 就删除”这条规则会误杀所有包含 JSON 或数学公式的页面，而”脏词列表”则可能把合法的医学或性教育内容一并排除。

一个有趣的附加实验是：T5 团队还做了一个 WebText-like 的子集，只保留出现在 OpenWebText 链接中的页面（即 Reddit karma ≥ 3 的外链）。用了 12 个 Common Crawl dump 才凑到 17GB——而原版 WebText 有 40GB，这说明 Common Crawl 的覆盖面虽广，但远非完整。

3.7.1 Worked Example：去重方法对比

去重是数据清洗中最关键的步骤之一。不同去重方法在精度、召回率和计算成本上差异显著：

方法	原理	优点	缺点	典型用户
精确去重	计算文档或段落的 SHA-256 哈希，完全匹配则删除	零误杀，实现简单	对细微修改无效（改一个字就不匹配）	CCNet
MinHash + LSH	用 n-gram 的最小哈希估计 Jaccard 相似度，将相似文档聚在同一桶内	能捕捉近重复（如换了标题或广告的相同文章）	有一定误杀率；超参选择影响大	RefinedWeb, FineWeb, SlimPajama
Bloom filter	用概率数据结构记录已见过的 n-gram 或 URL	内存高效	有假阳性	Dolma
Suffix array	找到精确重复的长字符串	可以做子文档级去重	计算成本高	The Pile

表 8: 去重方法对比：没有银弹，需要根据数据规模和质量目标选择

数据去重不充分会导致模型记忆

研究已经证明，训练数据中的重复内容会导致模型**逐字记忆**（verbatim memorization）。具体表现为：给模型一段重复出现过的文本的前缀，它能几乎完美地续写出后续内容。这不仅是隐私风险（如果训练数据包含个人信息），也是版权风险（如果模型能复述受版权保护的作品）。Carlini et al. (2023) 的研究表明，一段文本在训练集中出现的次数越多，模型能完整复述它的概率就越高。去重不是”锦上添花”，而是防止模型变成搜索引擎的基本功。

3.8 GPT-3、The Pile、Gopher、LLaMA：数据配方开始变成公开工程

GPT-3 使用 Common Crawl、WebText2、Books1/Books2 和 Wikipedia，约 400B tokens。其 Common Crawl 处理方式值得注意：训练了一个质量分类器，用 WebText、Wikipedia、Books1、Books2 作为正例，将其余 Common Crawl 内容作为负例，然后用这个分类器对整个 Common Crawl 做打分过滤。此外还做了模糊去重（包括与 WebText 和评测基准的去重）。

The Pile 则是开源社区对 GPT-3 封闭配方的回应，由大量志愿者在 Discord 上协调完成，汇聚 22 个高质量域，共 825GB 文本（约 275B tokens）。它包括 Pile-CC（使用 WARC + jusText 转换，优于 WET）、PubMed Central（500 万篇 NIH 资助的公开论文）、arXiv（使用 LaTeX 源码）、甚至安然公司邮件（500 名高管的 50 万封邮件，2002 年调查期间公开）。

Gopher 的 MassiveText 使用了 MassiveWeb（自建网页过滤）、C4、书籍、新闻、GitHub 和 Wikipedia，总计 10.5TB 文本，但 Gopher 实际只训练了 300B tokens（约 12%）。MassiveWeb 的过滤使用手工规则而非分类器（例如” 80% 的词至少包含一个字母字符”），并使用 Google SafeSearch 做毒性过滤（而非词表）。

LLaMA 的数据混合又进一步把” 多源拼接 + 质量过滤 + 近重复删除” 做成了工程常态，使用 CCNet 处理的 Common Crawl、C4、GitHub（只保留宽松许可证）、Wikipedia（20 种语言）、Project Gutenberg 和 Books3（来自 The Pile）、arXiv、Stack Exchange（28 个最大站点），共 1.2T tokens。

3.8.1 主要预训练数据集对比

数据集	年份	主要来源	Tokens	使用的模型	是否公开
WebText	2019	Reddit 外链网页	~10B	GPT-2	否
C4	2019	CC 单快照 + 规则	156B	T5	是
CCNet	2019	CC + Wiki 过滤	可变	XLNet, BERT	是
GPT-3 data	2020	CC + books + wiki	400B	GPT-3	否
The Pile	2021	22 域混合	275B	GPT-J, NeoX	是
MassiveText	2021	多源 + 规则过滤	10.5TB	Gopher	否
LLaMA data	2023	CCNet+C4+GitHub 等	1.2T	LLaMA	配方公开
RedPajama v1	2023	LLaMA 配方复现	1.2T	—	是
RefinedWeb	2023	CC + trafilatura	5T	Falcon	部分 (600B)
FineWeb	2024	95 个 CC 快照	15T	—	是
Dolma	2024	多源 (Reddit 等)	3T	OLMo	是
DCLM	2024	CC + 分类器过滤	3.8T	—	是
Nemotron-CC	2024	CC + 重写 + 集成	6.3T	Nemotron	部分

表 9: 主要预训练数据集对比：从 10B tokens 到 15T tokens，规模增长了三个数量级

3.8.2 数据过滤方法对比

过滤方法	原理	优点	缺点	代表
规则过滤	手写规则（长度、标点、脏词等）	透明、可解释、无偏差引入	误杀/漏网多，难以覆盖所有情况	C4, Gopher
Perplexity 过滤	过用 Wikipedia 训练 LM, 低 PPL 即高质量	有统计基础，自动化程度高	偏向 Wikipedia 风格，排斥口语和创意	CCNet
分类器过滤	训练二分类器区分“好/坏”文本	灵活，可定制正负例	正例选择决定偏见方向	DCLM, GPT-3
集成过滤	多个分类器/打分器投票	降低单一偏见	计算成本高	Nemotron-CC

表 10: 四种主要过滤方法各有适用场景，现代系统通常组合使用

为什么公开配方有价值

它让研究者能把”模型效果”拆成”数据选择”的结果，而不是只看最终指标。数据公开得越清楚，越能比较不同 filtering / mixing 策略的真实影响。值得注意的是，最新一代模型的训练 token 量已经远超上表：Llama 3 训练了 15T tokens，Qwen3 训练了 36T tokens。数据规模的增长速度甚至超过了模型参数的增长。

3.9 The Pile 的社区意义

The Pile 不是只是”另一个大语料”，而是开源社区把数据配方从私有资产转成公共议题的标志。它把书籍、论文、代码、百科、问答和其他域混在一起，让”开源模型的数据应该长什么样”第一次变成一个可讨论的问题。

The Pile 中几个值得关注的子集：

- **Project Gutenberg**: 始于 1971 年，截至 2025 年约有 7.5 万本书，主要是版权已过期的公共领域作品（莎士比亚、贝多芬等）。
- **Books3**: 来自影子图书馆 Bibliotik 的 19.6 万本书，包含 Stephen King、Zadie Smith 等当代作者的作品。因版权侵权已被下架。
- **StackExchange**: 用户贡献的问答数据，有声望系统和标签，Q&A 格式天然接近指令跟随场景。
- **GitHub**: 代码数据不仅有助于编程任务，folklore 认为也有助于推理能力。存在大量重复 (fork、复制的代码)。

影子图书馆与训练数据

Library Genesis (LibGen)、Z-Library、Anna's Archive、Sci-Hub 等影子图书馆绕过版权和付费墙，提供了海量书籍和论文。LibGen 有约 400 万本书（2019 年数据），Sci-Hub 有约 8800 万篇论文（2022 年数据）。这些资源在学术界和发展中国家有巨大需求，但在法律上争议极大。Meta 被曝出使用 LibGen 训练模型，引发了作者群体的集体诉讼。

3.10 RedPajama、RefinedWeb、FineWeb：网页数据进入精加工时代

LLaMA 的公开配方启发了 RedPajama 和 SlimPajama (Cerebras 对 RedPajama 做 MinHashLSH 去重后得到的 627B token 子集)。随后 RefinedWeb 和 FineWeb 又把网页数据处理进一步推进。

RefinedWeb 的核心观点是“web data is all you need”——只要清洗得够好，纯网页数据就能媲美多源混合。它使用 trafilatura 从 WARC 提取文本（而非依赖 WET），应用 Gopher 的规则过滤（有意避免 ML 过滤器以减少偏见），并用 MinHash 在 5-gram 上做模糊去重。最终发布了 600B tokens（总量 5T）。

FineWeb 最初是对 RefinedWeb 的复现尝试，但在过程中做了多项改进：处理了 95 个 Common Crawl 快照，加入了 URL 过滤、语言识别（英语概率 > 0.65）、Gopher + C4 规则过滤、MinHash 去重，以及邮箱和公共 IP 地址的匿名化（PII 保护）。最终产出 15T tokens。

网页数据工程的底层变化

早期重点是“尽量多抓”。后来重点变成“怎样抓得更像人想读的文本，而且不把隐私和噪声一起放大”。关键转折是：当 RefinedWeb 证明精心清洗的纯网页数据可以和多源混合相竞争时，整个社区开始认真对待数据清洗本身的工程价值。

3.11 Dolma、DCLM、Nemotron-CC：从多样性到可比较过滤

Dolma 进一步把 Reddit（来自 Pushshift 项目，2005–2023 年的帖子和评论）、学术论文（来自 Semantic Scholar 的 4000 万篇论文，项目名 PeS2o）、Wikipedia、C4、Project Gutenberg 等来源组合起来，强调多样性。其 Common Crawl 处理使用 fastText 做语言识别、Gopher + C4 规则过滤（有意避免模型过滤）、Jigsaw 分类器做毒性过滤、Bloom filter 做去重。总量 3T tokens。

DataComp-LM (DCLM) 则试图把数据处理算法变成标准化基准。它先把 Common Crawl 处理成 DCLM-pool (240T tokens)，然后用质量分类器过滤。分类器的训练非常值得注意：正例（200K 条）来自 OpenHermes-2.5（GPT-4 生成的指令数据）和 ELI5（Reddit 的 Explain Like I'm 5 子版块），负例（200K 条）来自 RefinedWeb 的随机样本。用 fastText 训练后，在整个 DCLM-pool 上运行，过滤后得到 3.8T tokens。实验表明，这种分类器过滤显著优于其他过滤方法。

Nemotron-CC 走得更激进：针对 FineWebEdu 和 DCLM 过滤太激进（删除 90% 以上数据）的问题，它用分类器集成（Nemotron-340B-instruct 打分蒸馏出的快速模型 + DCLM 分类器）做质量评估，同时对低质量数据用语言模型做重写（rephrase），对高质量数据用语言模型生成任务（QA 对、关键信息提取等）。最终产出 6.3T tokens（其中高质量子集 1.1T）。

项目	策略	优点	潜在代价
Dolma	多源混合	覆盖面广，便于做系统比较	来源繁杂，清洗标准必须很清楚
DCLM	标准化数据基准	让“数据处理”变成可比较问题	容易把基准本身当成最终目标
Nemotron-CC	过滤 + 重写 + 加权	能把高质量样本放大	生成式重写会引入模型偏差

表 11: 从多源混合到可比较过滤，再到生成式增强，每一步都在牺牲不同东西

讲者对这类工作的关注点并不只是“谁更大”，而是“谁更能说明数据方法本身”。一旦你开始把 filtering、ranking、rewriting、mixing 分成可研究的子问题，数据处理就不再只是前处理脚本，而是模型能力的重要组成部分。

数据集	过滤方式	讲者强调的取向
CCNet	Wikipedia 风格的 n-gram 打 分	质量优先，偏保守
C4	大量规则 + 英文过滤	简单透明，覆盖较广
The Pile	人工策划的多域混合	多样性优先
LLaMA / RedPajama	多源混合 + 清晰整理	开源可复现
FineWeb	更细致的去重、语言识别、PII 清理	现代网页数据工程
DCLM	数据竞赛基准 + 模型式过滤	方法可比较
Nemotron-CC	过滤 + 生成式重写 + 质量 加权	在高质量与规模之间折中

表 12: 不同代际的数据集各自偏好的质量与规模平衡点

3.12 为什么 open source 之后，配方会越来越像工程规范

开源模型的一个重要后果，是数据配方不再只属于单一实验室。任何人都可以比较：你是更多依赖网页、更多依赖百科，还是更多依赖代码和论文；你是保守过滤还是激进过滤；你是追求规模还是追求可复现。这个比较一旦成立，整个领域就会开始建立共同语言，比如 token 数、域占比、去重策略、语言识别和 PII 清理。

这也是为什么很多公开模型的说明文件越来越像系统设计文档，而不是简单的数据来源表。数据工程一旦变成公共话题，大家讨论的就不只是“用了什么”，还包括“为什么选这个”“怎么证明这个选择有效”。

3.13 代码数据的特殊地位

代码数据在预训练中有着超出“编程能力”的作用。Folklore 认为，代码训练有助于模型的**推理能力**——因为代码本身就是结构化的逻辑表达。GitHub 代码的处理有几个特殊挑战：

- **重复率极高**：fork、复制粘贴、自动生成的代码（如 protobuf、配置文件）使得原始 GitHub 数据中有大量近重复。The Stack 项目从 1.37 亿个仓库中克隆了 510 亿个文件，但去重后只有 50 亿个是唯一的。
- **许可证过滤**：The Stack 只保留宽松许可（MIT、Apache）的代码，使用 go-license-detector 工具自动识别。这将数据量从 TB 级压缩到 3.1TB。
- **不只是代码**：仓库中还有 README、issue、commit message、pull request 评论等元数据，这些对训练也有价值。GH Archive 以小时为单位记录 GitHub 事件快照。

3.14 预训练数据本章小结

预训练数据的演化可以概括为三个阶段：(1) 早期用少量高质量来源（Wikipedia + 书籍），(2) 中期转向大规模网页爬取加过滤，(3) 最新趋势是把过滤、去重、重写做成可比较的工程基准。贯穿始终的核心问题是：**如何在规模和质量之间找到最佳折中**。

4 中训练、后训练与能力定制

4.1 长上下文不是顺带出现的

长上下文不是 pre-training 的自然副产品，而常常需要单独训练阶段来扩展。讲者给出的关键词是：如果想把上下文长度从 4K 推到 100K 或更长，通常要在模型结构和数据上同时做文章。当前主流模型的上下文长度差异显著：

模型	上下文长度	备注
早期 GPT / BERT	512 – 2K tokens	受限于绝对位置编码
LLaMA 2	4K tokens	标准 RoPE
DeepSeek v3	128K tokens	专门的长上下文训练
Claude 3.5 Sonnet	200K tokens	需要特殊数据与注意力机制
Gemini 1.5 Pro	1.5M tokens	极长上下文，工程挑战巨大

表 13: 上下文长度从 512 到 150 万，增长了三个数量级

Transformer 的注意力机制对序列长度是二次方复杂度，因此直接在预训练阶段用长序列非常昂贵。更实际的做法是先用短上下文预训练，再用专门的长上下文数据做扩展训练。

长上下文为什么是数据问题

- 长文档比短句更难得，也更容易被过滤掉。
- 训练长上下文时，模型不仅需要长序列，还需要长文档的分布。
- 长上下文能力往往来自专门的数据混合，而不是“顺手训练出来”。

LongLoRA 之类的方法说明，长上下文能力通常要配合专门的训练文档，例如书籍、数学证明、长文档任务等。换句话说，数据的长度分布本身就是模型能力的一部分。

长上下文所需材料	为什么有用	常见缺口	工程后果
书籍 / 教材	连续章节、前后引用多	许可与格式清洗	能训练跨段依赖
数学 / 证明	需要跟踪符号和中间步骤	质量低但格式要求高	适合做推理长度扩展
长文档 QA	问题和证据跨很多段	需要精确对齐证据	强化检索与引用能力
代码仓库 / issues	上下文跨度长、结构化强	噪声与依赖复杂	帮助模型维持多轮状态

表 14: 长上下文不是“把窗口拉长”就结束了，还得有对应的数据形态

如果你只把上下文长度理解成一个超参数，就会误以为“把 RoPE 或 attention 改一下”就解决了问题。实际上，没有足够多的长文档样本，模型只是在更大的窗口里做更大的短文推断，长程能力不会自动冒出来。

4.2 从任务到 prompt

讲者把大量 NLP 训练数据看成一种统一的转换过程：把已有任务转成 prompt，让模型在同一接口上学习。这里最常见的来源是监督基准、问答数据和可模板化的任务集合。核心流程是：(1) 收集已有 NLP 任务，(2) 用模板将其转成 prompt-response 对，(3) 混合大量不同任务，(4) 让模型在统一接口下学习。

任务化数据的核心思路

1. 把分类、抽取、问答、阅读理解等任务统一成 prompt/response 形式。
2. 通过模板化，让模型学会在统一接口下完成不同任务。
3. 用多任务混合提升泛化，而不是只盯一个单点指标。

Super-Natural Instructions 和 Flan 2022 是这一路线的代表。它们证明了一个事实：大量现有 NLP 数据集并不需要重新发明，只需要被系统地改写成统一格式，就能变成更强的训练材料。

任务类型	如何 prompt 化	模型学到什么	常见失败
分类	给出标签选项，让模型选一个	输出格式约束	只会猜标签，不会解释
抽取	让模型从文本中摘出答案片段	定位证据	过度生成、漏掉边界
阅读理解	把问题和上下文放进 prompt	跨句整合	依赖模板，换写法就掉点
问答 / 指令	让模型直接回答人类请求	交互接口	忽略上下文或胡乱拒答

表 15: 把任务统一成 prompt-response 后，训练目标就从“识别任务”变成“使用统一接口”

这一层变化很重要，因为它让模型在训练时就接触到“用户会怎么提问”的结构。后训练的很

多收益，不是来自新知识，而是来自对输入形式和输出形式的再组织。

4.3 Instruction tuning 和 chat data

指令数据和聊天数据是 post-training 的核心。预训练模型已经学会语言分布，但还不会自然地回答“请你帮我做什么”。因此需要把自然语言任务转成显式的指令-响应格式。指令数据的演化经历了从人工标注到大规模合成的过程：

数据集	规模	来源	年份	训练的模型
Alpaca	52K	text-davinci-003 自动生成 (self-instruct)	2023	LLaMA 7B
Vicuna	70K	ShareGPT 真实用户对话	2023	LLaMA 13B
Baize	111.5K	GPT-3.5 自聊 (种子来自 Quora/SO)	2023	LLaMA
WizardLM	–	Evol-Instruct (渐进增加难度)	2023	LLaMA
MAmmoTH2	10M	Common Crawl 中的 quiz 站点 + GPT-4 提取 QA	2024	Mistral 7B
OpenHermes	1M	多源聚合, GPT-4 生成	2024	Mistral 7B
Llama 2 chat	27.5K	高质量供应商标注	2023	Llama 2
Nemotron PT	–	公开 prompt + 合成 response (Llama/Mixtral/DeepSeek)	2024	Llama-Nemotron

表 16: 后训练数据演化：从 27.5K 人工标注到百万级合成数据

这些方法在做什么

- **Alpaca**: 用强模型 (text-davinci-003) 通过 self-instruct 方法自动生成 52K 指令样本, 让 LLaMA 7B 学会跟随指令。成本极低 (约 \$500), 但质量受限于生成模型。
- **Vicuna**: 利用 ShareGPT 上真实用户与 ChatGPT 的对话 (70K 条), 逼近真实聊天分布。优势在于数据反映了用户真实需求, 但 ShareGPT 已停运。
- **Baize / WizardLM**: Baize 用 GPT-3.5 自聊生成多轮对话; WizardLM 用 Evol-Instruct 渐进增加问题难度和广度。
- **MAmmoTH2**: 从 Common Crawl 中用 fastText 分类器找出 quiz 类网站, 再用 GPT-4 和 Mixtral 提取 QA 对, 得到 1000 万条指令, 显著提升数学能力。
- **OpenHermes / Nemotron**: 把多源指令、合成回答和推理轨迹再加工成更大训练集。Nemotron 特别注意使用商业可行的模型 (Llama、Mixtral、DeepSeek r1、Qwen) 生成回答, 避免依赖 GPT-4 带来的许可限制。

Llama 2 的反直觉发现

Llama 2 chat 的 SFT 只用了 27,540 条高质量供应商标注数据, 但团队声称这比使用开源社区的数百万条数据效果更好。他们的反思是: 与其把预算花在标注更多 SFT 数据上, 不如把更多精力放在 RLHF 数据的收集上。这说明后训练数据的**质量远比数量重要**——少量精心标注的数据可以胜过大量噪声数据。

4.3.1 后训练数据策略对比

策略	SFT 数据	RLHF/偏好	代表	核心理念
人工标注	数万条	数十万偏好对	Llama 2 chat	少而精，质量为王
合成蒸馏	数万-数百万	模型自动排序	Alpaca, Open-Hermes	用强模型教弱模型
自聊/自演化	自动生成	自动生成	Baize, WizardLM	低成本扩大覆盖面
网页挖掘	数百万	-	MAmmoTH2	互联网本身有大量隐含指令
混合策略	多源聚合	多源聚合	Nemotron, Tulu	取长补短

表 17: 后训练数据策略：没有单一最优方案，工业界普遍采用混合策略

合成数据的双刃剑

合成数据可以补足稀缺能力，但也会把生成模型自己的偏差、幻觉和风格一起放大。它不是免费午餐，必须依赖过滤和验证。

环节	典型做法	质量控制点	常见问题
生成题目	用强模型扩题、改题、变难度	覆盖度、难度分布	容易跑偏到无关问题
生成答案	多候选回答、CoT 草稿	正确性、格式一致性	幻觉和不稳定风格
筛选排序	规则 + 模型打分 + 人工复核	一致性、去重、难例保留	评分器偏见
回流训练	把高分样本加入训练集	避免风格塌缩	过拟合合成痕迹

表 18: 合成数据真正难的地方在筛选和回流，不在“生成”本身

合成数据之所以越来越重要，是因为很多稀缺能力的原始分布本来就很难直接从互联网抓到。问题不是“能不能生成”，而是“生成以后你有没有能力判断哪些样本真的在帮模型变强”。这要求你同时懂任务、懂评价、懂过滤和懂训练循环。

4.4 为什么会越来越依赖合成数据

讲者不断强调一个趋势：很多能力已经不能只靠“抓互联网”得到，尤其是数学、代码、长上下文、推理和安全对齐。于是人们开始利用更强模型生成题目、生成答案、生成中间推理轨迹，再把这些合成样本重新喂给训练系统。

合成数据的基本闭环包含四步：

- 1. **种子问题/提示**：从已有数据集或人工设计中获取初始问题。
- 2. **强模型生成**：用更强的模型（如 GPT-4）生成答案或推理轨迹。

3. **过滤/验证/排序**：用规则、模型打分或人工审核筛选高质量样本。
4. **回流训练**：将筛选后的样本加入弱模型的训练集。

这个闭环可以迭代进行：弱模型训练后变强，又可以生成更好的数据，形成正反馈循环。但也存在风险——如果过滤不严，模型可能越来越收敛到生成模型的特定风格（“模型塌缩”）。

4.5 中训练与后训练本章小结

中训练和后训练的核心区别在于数据的性质和规模：

- **中训练**主要用更高质量的网页数据、数学数据、代码数据和长文档来补强预训练模型的能力短板，规模通常在百亿到万亿 token 量级。
- **后训练**用指令数据、对话数据和偏好数据把模型变成可交互的助手，规模通常在数万到百万样本量级。
- 合成数据在两个阶段都越来越重要，但后训练对合成数据的依赖更强——因为高质量的人类对话数据天然稀缺且获取成本极高。
- 质量在后训练中远比数量重要：Llama 2 用 2.7 万条精标数据胜过百万条开源数据，说明了这一点。

5 法律、许可与 fair use

讲者把版权问题放在数据章节里，不是因为它只是法律附录，而是因为它直接影响哪些数据可以进入训练流水线。当前围绕生成式 AI 有大量诉讼，绝大多数与版权相关。

5.1 知识产权法基础

知识产权法的目标是**激励**知识产品的创造。它包含四种主要类型：版权（copyright）、专利（patent）、商标（trademark）和商业秘密（trade secret）。对语言模型训练最相关的是版权。

版权法的关键特征：

- **历史**：可追溯到 1709 年英国的安妮法令（Statute of Anne），美国最新的版本是 1976 年版权法。
- **保护对象**：保护“固定在任何有形表达媒介中的原创作品”。注意保护的是**表达**（expression），而非**想法**（idea）——快速排序的想法不受保护，但某段特定的快速排序代码受保护。
- **门槛极低**：不需要注册即可获得版权保护（与专利不同）。你的网站、博客文章、社交媒体帖子都自动受版权保护。
- **注册的作用**：虽然不需要注册就有版权，但要**起诉侵权**必须先注册（注册费 \$65）。
- **期限**：通常 75 年后过期，进入公共领域（这就是 Project Gutenberg 能合法提供莎士比亚、贝多芬作品的原因）。

核心事实：互联网上大多数内容都受版权保护

由于版权门槛极低且无需注册，互联网上几乎所有内容——新闻文章、博客帖子、论坛评论、代码仓库、图片——默认都受版权保护。”能公开访问”绝不等于”可自由使用”。训练语言模型的第一步就是复制数据，这本身就构成版权法意义上的”复制”行为。

5.2 为什么版权会卡住数据

- 互联网内容大多数并不属于公共领域。
- 训练模型第一步就是复制数据到存储和处理系统里，这本身就涉及版权复制问题。
- 模型输出会影响原内容的市场，因此争议不只是”有没有拷贝”，还包括”有没有替代市场”。
- 版权不仅仅关于逐字复制——角色、情节（如 Harry Potter）也可受版权保护。

使用受版权保护的作品有两条合法路径：(1) 获取许可/授权，(2) 诉诸合理使用（fair use）条款。

5.3 许可、CC 协议和训练授权

一条更稳妥的路线是通过许可协议获取训练权。许可证本质上是”承诺不起诉”。CC 协议（Creative Commons）由 Lessig 和 Eldred 于 2001 年创建，旨在弥合公共领域和现有版权之间的鸿沟，使大量内容可在一定条件下自由分发。

授权方式	典型案例	对训练的影响
CC 协议	Wikipedia、Open Courseware、Khan Academy	允许再分发，但需注意 NC/ND 限制
平台合作	Google-Reddit、OpenAI-Shutterstock、OpenAI-StackExchange	付费获取训练权
公共领域	Project Gutenberg（版权过期作品）	完全自由使用
宽松开源许可	MIT/Apache 代码（如 The Stack）	允许使用但需保留许可声明

表 19: 数据授权路线：从免费的公共领域到付费的平台合作

数据供应链越来越像传统内容产业的授权链条。越来越多的模型开发者开始付费获取训练数据授权——这既是法律风险管理，也反映了高质量数据的稀缺性。

5.4 fair use 的四个因素

美国法下的 fair use 不是一句笼统口号，而是四个因素的综合判断：

1. **使用目的和性质**：是否具有变换性（transformative）、是否是商业用途。教育用途优于商业用途，变换性使用优于复制性使用。
2. **原作品的性质**：事实性作品（如电话簿）通常比创作性作品（如小说）更容易被认为 fair use。

3. **使用数量和实质性**：是否拿了过多原作品。使用片段比使用全文更有利。
4. **对市场的影响**：是否替代了原作品的商业价值。这在 AI 领域争议最大——语言模型可能替代作家、艺术家的市场。

Fair use 的一些已有判例可供参考：

- **Google Books 案** (Authors Guild v. Google, 2002–2013)：Google 对书籍建索引并展示片段被判为 fair use。
- **模仿** (parody) 通常被认为是 fair use。
- 看电影后写摘要属于 fair use；重新实现算法 (idea) 而非复制代码 (expression) 属于 fair use。

对大模型训练而言，关键争论点包括：

- 复制数据（训练的第一步）本身就是版权法意义上的“复制”，即使最终不输出原文。
- 训练过程是高度变换性的——远非简单的复制粘贴。
- ML 系统关注的是想法（如“停车标志长什么样”），而非具体表达（某张特定的停车标志照片的艺术选择）。
- 但无论版权法如何判定，语言模型**确实会影响**作家和艺术家的市场。

常见误区

很多人会把“fair use 可能成立”误认为“我就可以随便抓”。这不成立。ToS、robots.txt、合同义务、隐私和反爬策略都可能单独构成约束。

层次	看什么	回答的问题	不能替代什么
License / CC	许可证条款	你有没有被授权使用	不能替代具体的使用场景判断
Fair use	美国法上的合理使用因素	在某些条件下能否免责	不能替代合同和平台规则
ToS / robots.txt	平台自己的访问规则	你能不能抓、怎么抓	不能替代版权法分析
Privacy / contract	隐私、保密、合同义务	你是否违反额外承诺	不能靠“公开可见”自动规避

表 20: 训练数据合规是多层约束，不是单一法律问题

5.5 TOS 为什么也重要

即使某些行为可能被版权法容纳，平台的 terms of service 仍然可能额外限制你下载、抓取或再分发数据。例如，YouTube 的 ToS 明确禁止下载视频，即使视频本身使用 CC 许可。对训练系统来说，合规不是只看版权法条，还要看平台规则、隐私政策和爬虫约束。

合规三件套

- **License**: 你有没有被允许用。
- **Fair use**: 在美国法下是否可能构成合理使用。
- **Terms / robots.txt**: 平台自己是否额外限制抓取和再用。

实际操作中，这三者必须逐层检查，且任何一层的否定都足以构成约束。合规决策是一个保守取交集的过程，而非选最宽松的一层来执行。

5.6 拓展阅读

版权与 AI 训练是一个快速发展的领域。以下资源提供了更深入的分析：

- Stanford CS324 课程笔记中的 legality 章节
- Lemley & Casey, *Fair Learning* (Texas Law Review)
- Henderson et al., *Foundation Models and Fair Use* (2023)
- Grimmelmann, *The Files are in the Computer* (2024)

5.7 把整堂课串起来：一个端到端的数据管线

如果把这一讲压缩成一个最小的工程流程，它大概长这样：先定义目标能力，再找合适来源，然后做清洗、去重、过滤、混合、合成、再训练，最后再检查合规和输出形式。所谓“数据工程”，不是任何一环单独做好，而是整条链路都不能塌。

管线阶段	输入	输出	关键问题
Source selection	网页、书籍、百科、代码、论文	候选语料池	这个来源是否真的适合目标能力
Cleaning / filtering	原始文本与 HTML	更干净的纯文本	噪声、重复、语言偏差如何控制
Mixing / weighting	多个过滤后的子集	训练配方	质量与多样性如何平衡
Promptification	任务集、问答集、基准	prompt-response 数据	如何把不同任务统一成一个接口
Post-training	指令、聊天、偏好、合成样本	Instruct / chat 模型	如何把模型变成可用助手
Governance	许可证、ToS、隐私、版权	合规决策	训练前是否已经越线

表 21: 一个完整的数据工程流程必须同时处理能力目标与合规目标

这张表的意义是：数据不是单一的“预训练料”，而是从来源、清洗、配方、任务化、对齐到合规的一整条供应链。真正成熟的团队，往往不是只会抓更多数据，而是知道在哪一步该停、在哪一步该换来源、在哪一步该退回重新设计目标。

5.8 一个更具体的例子：把网页做成训练集

假设你现在要做一个“更像 WebText / C4 / FineWeb 的数据集”。正确的问题不是“怎么抓最多网页”，而是“怎样把抓回来的网页一步步变成可训练、可解释、可维护的文本”。这个过程通常是：

- 1. 先按域名、语言和站点信誉做初筛，避免把垃圾和明显非目标语言混进来。
- 2. 再对 HTML 做清洗，提取正文、标题、段落和有意义的结构。
- 3. 接着做去重和近重复检测，防止同一段模板被反复喂给模型。
- 4. 然后做质量打分，把更接近自然语言、信息密度更高的文本留下。
- 5. 最后把它们和其他来源混合，并为后训练单独留出高质量子集。

步骤	如果做得好	如果做得差	讲者会怎么评价
初筛	噪声被前置排掉	低质页一开始就污染全流程	先别贪多,先看干净度
正文抽取	保留可读文本和结构	只剩碎片、导航和 boilerplate	HTML 不是文本，别直接信它
去重	训练 token 更有效	模型不断背重复页	去重是基本功,不是附加项
质量打分	留下更像人话的文本	把稀有但有用的长尾误杀掉	要有代理分布意识
混合与留档	便于复现与后续调参	日后很难知道为什么有效	配方管理本身是研究对象

表 22: 网页语料工程的关键，不是某个单点技巧，而是端到端的可控性

如果把这个例子说得更直白一点：网页数据工程其实是在做“把自然世界里的脏文本，变成统计学习能够消费的干净样本”的工作。你不是在整理网页，你是在重建一个适合训练的文本分布。

6 总结与延伸

这一讲的目标，其实是把数据从”看不见的前置条件”变成”显式的工程对象”。数据不是被动存在的输入，而是可以通过来源选择、过滤、混合、再写、再对齐被系统性塑造的。

最后的结论

1. 数据工作本身就是一个长期、并行、可扩展、且高度依赖经验的工程。
2. 预训练、mid-training、post-training 的数据目标不同，不能混为一谈。
3. 网页语料不是天然可用，必须经过抓取、清洗、去重、质量打分和合规检查。
4. 任务化数据、指令数据和合成数据已经成为后训练的重要组成。
5. 版权和平台条款不是边角问题，而是训练数据供应链的一部分。
6. 数据管线中的大量步骤仍然是启发式的，有大量改进空间。

讲者想传达的最朴素一句话是：**模型能力不是”长出来”的，而是”喂出来”的**。你怎么喂，决定你得到什么样的模型。

6.1 本讲的核心问题链

回顾这一讲的逻辑链条：

1. **我们用什么数据训练？**从 Wikipedia + 书籍到数十万亿 token 的网页数据，来源越来越广、清洗越来越精。
2. **我们如何过滤、混合和验证数据？**从简单规则到分类器、从精确去重到模糊去重、从人工策划到基准化评测。
3. **有什么法律和伦理约束？**版权、许可、ToS、隐私——每一层都可能独立构成限制。
4. **这些选择如何变成你得到的模型？**数据配方决定模型能力分布，数据质量决定模型能力上限。

6.2 拓展阅读

- Stanford CS324 课程中的 data 和 legality 章节提供了更多细节
- Longpre et al., *A Pretrainer's Guide to Training Data* (2023) 系统梳理了预训练数据的各个方面
- Penedo et al., *The FineWeb Datasets* (2024) 是现代网页数据清洗的最佳实践参考
- Li et al., *DataComp-LM* (2024) 定义了数据处理方法的标准化评测框架